

软件版权保护基本技术

4 软件加壳

- 1) 加壳是指，在原二进制文件（如可执行文件、动态链接库）上附加一段专门负责保护该文件不被反编译或非法修改的代码或数据，以对源文件进行加密或压缩，并修改原文件的运行参数或运行流程，使其被加载到内存中执行时，附加的这段代码—保护壳，先于原程序运行，执行过程中先对原程序文件进行解密和还原，完成后再将控制权转交给原程序。加壳后的程序能够增加逆向（静态）分析和非法修改的难度。
- 2) 根据对原程序实施保护方式的不同，壳大致可以分为两类：压缩保护型壳和加密保护型壳。



CONTENTS



压缩壳



保护壳



实例分析



压 缩 壳





壳

- 壳是软件逆向分析的常见主题，为了理解好它，需要掌握有关PE文件格式、操作系统的基本知识（进程、内存、DLL等），同时也要了解有关压缩/解压缩算法的基本内容。
- 壳可以分为两类：
 - 1、压缩壳；
 - 2、保护壳。



压缩壳

- 为了减少可执行文件的体积，需要对可执行文件进行压缩，可执行文件经过压缩后成为另一个可执行文件。
 - 1、内部含有解压缩代码，文件在运行瞬间于内存中解压缩后执行；
 - 2、程序的功能完全一样。



压缩的目的

- **缩减PE文件的大小**
 - 文件尺寸小是其突出的优点之一，更便于网络传输与保存；
- **隐藏PE文件内部代码与资源**
 - 使用压缩壳的另一个原因在于它可以隐藏PE文件内的代码及资源（字符串、API名称字符串）等。压缩后的数据以难以辨识的二进制文件保存，从文件本身看，这能有效隐藏内部代码与资源（当然解压缩后可以通过内存的Dump窗口查看）。



使用现状

- 运行时压缩的概念早在DOS时代就出现了，可当时并未广泛使用。因为那时的PC速度不怎么快，每次执行文件时，解压缩的过程会引起很大的系统开销。而现在的PC速度已经变得非常快，用户不能明显察觉运行时压缩文件与源文件在执行时间上的差别。



压缩壳种类

- PE压缩壳大致可分为两类：
 - 一类是单纯用于压缩普通PE文件的压缩壳，
如：UPX、ASPack;
 - 另一类是对源文件进行较大变形、严重破坏PE头的压缩壳。
经过此类压缩壳压缩后，PE文件难以通过工具进行正常辨识、对逆向造成阻碍，或用来进行软件的保护，或被恶意程序用来加大安全人员分析难度，
如：Upack、PESpin、NSAnti。



保护壳





保护壳

- PE保护壳是一类**保护PE文件免受代码逆向分析**的实用程序。它们不像普通的压缩壳一样仅对PE文件进行运行时压缩，而应用了多种防止代码逆向分析的技术（反调试、反模拟、代码混乱、多态代码、垃圾代码、调试器监控等）；
- 使用这些保护壳处理过的PE文件其尺寸反而要比之前的尺寸大些，详细分析保护壳需要丰富的逆向分析经验。



保护壳的使用目的

- **防止破解;**
 - 使用保护壳可有效保护PE文件，使其防止被非法破解并使用;
- **保护代码与资源**
 - 保护壳不仅可以保护PE文件本身，还可在文件运行时保护进程内存，防止程序所使用的资源通过内存被dump。因此，使用保护壳可以比较安全地保护程序自身的代码与资源。



使用现状

- 这类保护壳大量应用于对破解很敏感安全程序。比如安装在线游戏时会自动安装保护程序，游戏安全保护程序就是为了防止游戏被破解而存在的
 - 恶意游戏破解者总是想方设法破解游戏的安全保护程序，因为破解成功后他们可以进而制作各种外挂牟取暴利。为此，安全保护程序为了防止恶意破解会使用各种保护壳来保护自己。
- 另一方面，恶意代码也大量使用保护壳来防止（或降低）杀毒软件的检测。有些保护壳还能提供运行时改编代码，每次都会生成内容不同（但功能相同）的汇编代码，这给病毒诊断带来很大困难。



保护壳种类

- 保护壳种类多样，有公用程序、商业程序，还有专门供恶意代码使用的保护壳
 - 商用保护壳：ASProtect、Themida、SVKP等
 - 公用保护壳：UltraProtect、Morphine等
- 压缩壳与保护壳在代码逆向分析学习中占有非常重要的地位。分析压缩壳与保护壳本身也能学到很多，对提高逆向分析技术、有很大帮助。保护壳中使用的反调试技术往往水平非常高，需要具备关于操作系统和CPU的高级知识。



3

实例分析



壳的加载过程

- 保存入口参数
- 获取壳自己所需要使用的API地址
- 解密原程序各个区块的数据
- IAT的初始化
- 重定位项的处理
- HOOK-API
- 跳到程序原入口点（OEP）

- 保存入口参数

加壳程序初始化时保存各个寄存器的值，外壳程序执行完毕，再恢复各个寄存器的值，通常用pushad/popad,pushfd/popfd指令来对保护与恢复现场环境

- 获取壳自己所需要使用的API地址

一般外壳的输入表主要是GetProcAddress, GetModuleHandle和LoadLibrary这几个API函数

LoadLibrary: 将DLL文件映像映射到调用进程的地址空间中

GetModuleHandle: 获得DLL模块的句柄

GetProcAddress: 获得函数的地址

看C代码

- 解密原程序各个区块的数据

按照区块加密按照区块解密

- IAT的初始化

加壳时，壳会自己构造一个输入表，并将PE头中的输入表指针指向自建的输入表。PE装载器对自建的输入表进行填写，原来的输入表由壳来填写。

壳将新输入表结构从头到尾扫描一遍，对每一个DLL引入的所有函数获取地址，填入IAT表中

- 重定位项的处理

加壳的DLL比加壳的EXE修正时多一个重定位表

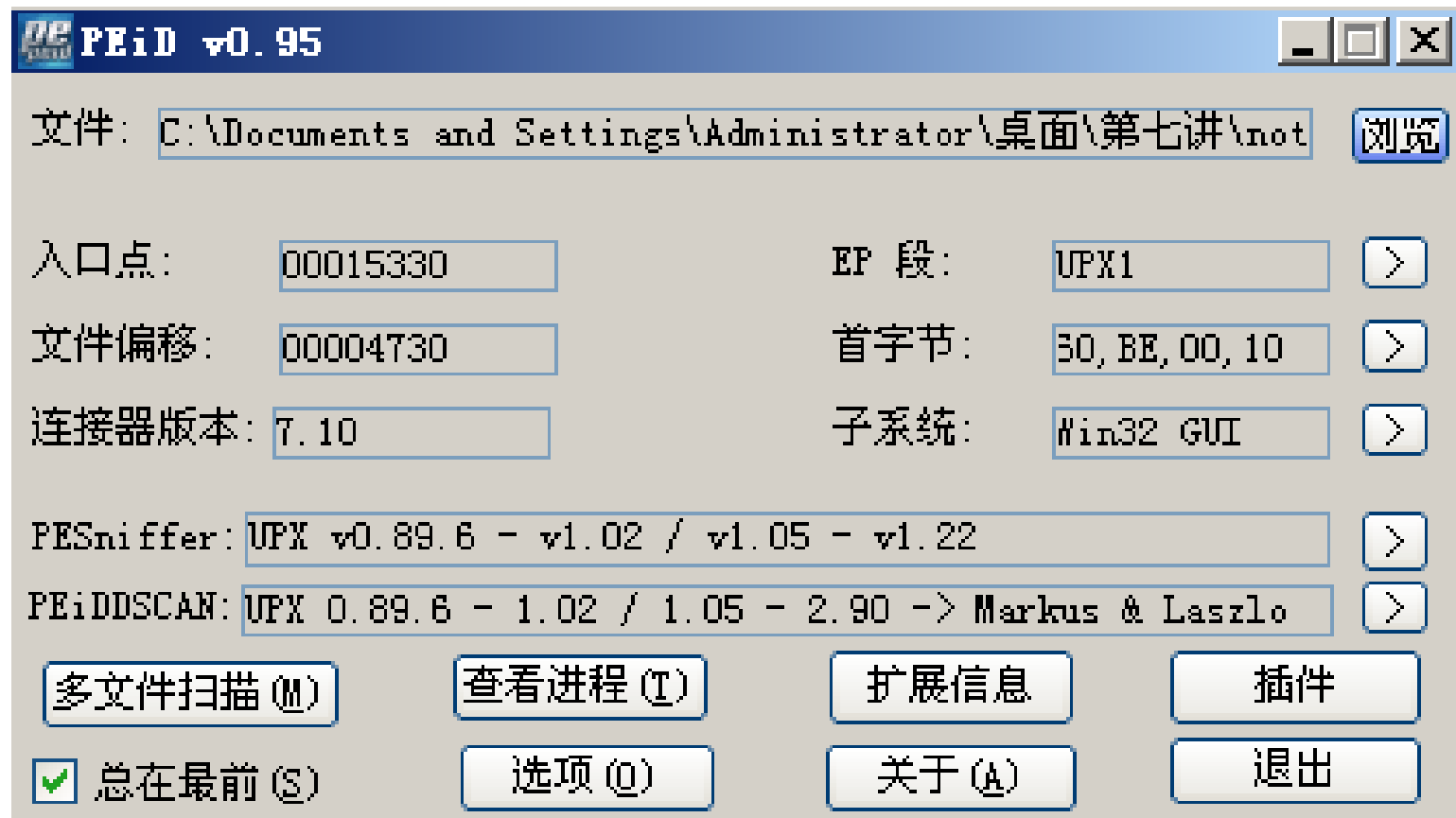
- HOOK-API

壳一般都修改了原程序的输入表，自己模仿装载器来填充输入表。在填充过程中，外壳就可以填充HOOK-API的代码地址，间接获得程序的控制权

- 跳到程序原入口点 (OEP)

脱壳

- 1.用脱壳机



- 2.手动脱壳

- 手动脱壳的常规步骤如下:
 - (1)、查找程序真正的入口点
 - (2)、抓取内存映像文件
 - (3)、重建PE文件（输入表）

手动脱壳

- **方法一：单步跟踪法**（用F8、F7、F4），一直跟踪到程序大跳转时，开始dump内存。主要用于比较简单的壳。
- **方法二：ESP定律法**（根据堆栈平衡原理找OEP），这是因为外壳程序在开始运行时需要保护现场，结束后恢复现场，从而保持堆栈平衡。

脱壳方式：在外壳执行完pushad或pushfd后，对ESP指向的内存地址设置硬件访问断点，然后F9运行程序，很快就可达OEP

手动脱壳

- **方法三：利用内存访问断点找OEP。**

内存断点：就是通过将需要设置断点的内存页设置为不可访问属性，这样，当程序访问相关内存地址时就会发生异常，从而中断下来。一般分为内存访问和内存写入两种断点。（以页为单位）

两次内存访问一次性断点法：通过两次内存断点的方式来找到OEP。

在第一个.text段处设置断点，然后F9运行，运行到断点之后F8单步跟踪，即可找到程序的真正入口点，然后脱壳。

- **方法四：根据编译语言寻找OEP。（查找函数，在函数前下断进行脱壳）**

函数调用：GetCommandLineA、GetModuleHandleA、GetVersion、GetStartupInfoA



使用UPX对程序进行压缩

- UPX是一个开源的、支持多种CPU的压缩壳
- 官方网站: <http://upx.github.io/>
- 使用方法
 - upx.exe要压缩的PE文件
 - UPX也能对加过UPX壳的PE文件进行脱壳

C:\WINDOWS\system32\cmd.exe

C:\Documents and Settings\Administrator\桌面\第七讲>upx

Ultimate Packer for eXecutables

Copyright (C) 1996 - 2008

UPX 3.03w Markus Oberhumer, Laszlo Molnar & John Reiser Apr 27th 2008

Usage: upx [-123456789dlthUL] [-qvfk] [-o file] file..

Commands:

-1	compress faster	-9	compress better
-d	decompress	-l	list compressed file
-t	test compressed file	-U	display version number
-h	give more help	-L	display software license

Options:

-q	be quiet	-v	be verbose
-oFILE	write output to 'FILE'		
-f	force compression of suspicious files		
-k	keep backup files		

file.. executables to (de)compress

Type 'upx --help' for more detailed help.

UPX comes with ABSOLUTELY NO WARRANTY; for details visit <http://upx.sf.net>

```
C:\Documents and Settings\Administrator\桌面\第七讲>upx notepad.exe -o notepad_111.exe
```

Ultimate Packer for eXecutables

Copyright (C) 1996 - 2008

UPX 3.03w Markus Oberhumer, Laszlo Molnar & John Reiser Apr 27th 2008

File size	Ratio	Format	Name
-----	-----	-----	-----
67584 -> 48128	71.21%	win32/pe	notepad_111.exe

Packed 1 file.

```
C:\Documents and Settings\Administrator\桌面\第七讲>
```

PE 编辑器

标志字: 010B RVA 数及大小: 00000010

区段表

名称	VOffset	VSize	ROffset	RSize	标志
.text	00001000	00007748	00000400	00007800	60000020
.data	00009000	00001BA8	00007C00	00000800	C0000040
.rsrc	0000B000	00008304	00008400	00008400	40000040

PE 编辑器

标志字: 010B RVA 数及大小: 00000010

区段表

名称	VOffset	VSize	ROffset	RSize	标志
UPX0	00001000	00010000	00000400	00000000	E0000080
UPX1	00011000	00005000	00000400	00004600	E0000040
.rsrc	00016000	00008000	00004A00	00007200	C0000040

可以看到第一个节区的 RawDataSize=0, 但是 VirtualSize=00010000H, 即文件中的大小为0, 虚拟内存中大小变为10000H。这就是说, 程序运行前, 解压缩代码与压缩的源代码都在第二个节区。文件运行时首先执行解压缩代码, 把处于压缩状态的源代码解压到第一个节区, 解压过程结束后即运行源文件的EP代码。另外可以看到EP代码位于压缩后文件的第二个节区(原文件在第一个)。资源节区大小几乎没有变化。PE头大小也一样。



使用UPX对程序进行压缩

notepad.exe

00000000	DOS头
00000040	DOS存根
000000E0	NT头
000001D8	节区头(".text")
00000200	节区头(".data")
00000228	节区头(".rsrc")
	NULL
00000400	
7600	节区(".text")
	NULL
00007C00	800 节区(".data")
	NULL
00008400	
8400	节区(".rsrc")

notepad_upx.exe

00000000	DOS头
00000040	DOS存根
000000E0	NT头
000001D8	节区头(".UPX0")
00000200	节区头(".UPX1")
00000228	节区头(".rsrc")
	NULL
00000400	0 节区(".UPX0")
00000400	4600 节区(".UPX1")
	NULL
00004A00	
7200	节区(".rsrc")
	NULL
0000BC00	

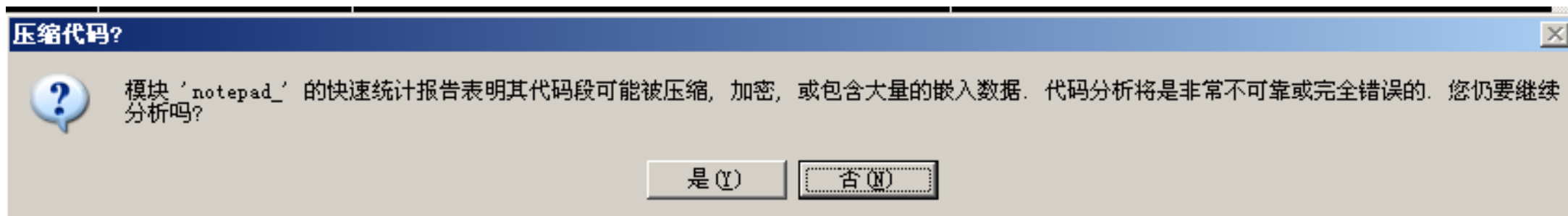
0100739D	\$ 6A 70	push 70	
0100739F	. 68 98180001	push notepad.01001898	
010073A4	. E8 BF010000	call notepad.01007568	
010073A9	. 33DB	xor ebx,ebx	
010073AB	. 53	push ebx	
010073AC	. 8B3D CC100000	mov edi,dword ptr ds:[&KERNEL32.GetModuleHandleA]	pModule = "" kernel32.GetModuleHandleA
010073B2	. FFD7	call edi	GetModuleHandleA
010073B4	. 66:8138 4D5A	cmp word ptr ds:[eax],5A4D 比较MZ签名	调用API
010073B9	. 75 1F	jnz short notepad.010073DA	
010073BB	. 8B48 3C	mov ecx,dword ptr ds:[eax+3C]	
010073BE	. 03C8	add ecx,ecx	
010073C0	. 8139 50450000	cmp dword ptr ds:[ecx],4550 比较PE签名	
010073C6	. 75 12	jnz short notepad.010073DA	
010073C8	. 0FB741 18	movzx eax,word ptr ds:[ecx+18]	
010073CC	. 3D 0B010000	cmp eax,10B	
010073D1	. 74 1F	je short notepad.010073F2	

https://blog.csdn.net/diamond_biu

未加壳的notepad.exe,可以看到在EP处调用了GetModuleHandleA()API, 获取程序的ImageBase (01000000) 。然后又进行了MZ、PE签名的比较。

notepad_upx.exe的EP代码

使用OD打开时弹出警告消息框，调试器判断文件为压缩文件。



Address	Disassembly	Comment
01015330	\$ 60 pushad	
01015331	- BE 00100101 mov esi,notepad_.01011000	ESI=01011000
01015336	- 8DBE 0000FFFF lea edi,dword ptr ds:[esi+FFFF0000]	
0101533C	- 57 push edi	EDI=01001000
0101533D	- 83CD FF or ebp,FFFFFFFF	
01015340	- EB 10 jmp short notepad_.01015352	
01015342	90 nop	
01015343	90 nop	
01015344	90 nop	
01015345	90 nop	
01015346	90 nop	
01015347	90 nop	
01015348	> 8A06 mov al,byte ptr ds:[esi]	
0101534A	- 46 inc esi	
0101534B	- 8807 mov byte ptr ds:[edi],al	
0101534D	- 47 inc edi	

https://blog.csdn.net/diamond_biu

PUSHAD命令将EAX~EDI寄存器的值保存到栈中，然后分别把第二个节区的起始地址（01011000）与第一个节区的起始地址（01001000）设置到ESI与EDI寄存器中。UPX文件第一节区仅存在于内存，该处即是解压后保存源文件代码的地方。

注：调试时像这样同时设置ESI和EDI，就能预见从ESI（Source）所指的缓冲区到EDI（Destination）所指的缓冲区的内存发生了复制。

跟踪UPX文件

遇到循环时，先了解作用再跳出。

循环1

在EP处执行Ctrl+F8开始跟踪(停止按F7\F8)，遇到第一个短循环。

暂停

地址	汇编指令	注释
010153D8	83FD FC	cmp ebp, -0x4
010153DB	76 0F	jbe short notepad_.010153EC
010153DD	> 8A02	mov al, byte ptr ds:[edx]
010153DF	42	inc edx
010153E0	8807	mov byte ptr ds:[edi], al
010153E2	47	inc edi
010153E3	49	dec ecx
010153E4	^ 75 F7	jnz short notepad_.010153DD
010153E6	^ E9 63FFFFFF	jmp notepad_.0101534E
010153EB	90	nop
010153EC	> 8B02	mov eax, dword ptr ds:[edx]
010153EE	83C2 04	add edx, 0x4
010153F1	8907	mov dword ptr ds:[edi], eax
010153F3	83C7 04	add edi, 0x4
010153F6	83E9 04	sub ecx, 0x4
010153F9	^ 77 F1	ja short notepad_.010153EC
010153FB	81CF	add edi, ecx

寄存器 (FPU)

寄存器	值	注释
EAX	FFFFFFFF00	
ECX	0000036A	
EDX	01001001	notepad_.01001001
EBX	DB800000	
ESP	0006FFA0	
EBP	FFFFFFFF	
ESI	01011005	notepad_.01011005
EDI	01001002	notepad_.01001002
EIP	010153E4	notepad_.010153E4
C	0	ES 0023 32位 0(FFFFFFFF)
P	1	CS 001B 32位 0(FFFFFFFF)
A	0	SS 0023 32位 0(FFFFFFFF)
Z	0	DS 0023 32位 0(FFFFFFFF)
S	0	FS 003B 32位 7FFDD000
T	0	GS 0000 NULL
D	0	
O	0	LastErr ERROR_NO_IMP

跳转已实现
010153DD=notepad_.010153DD

功能是从EDX中读取一个字节写入EDI中（此处EDI=EDX+1），EDX初始值为01001000，该区域均为NULL，因此该段循环没有什么实际意义，跳出（010153E6处F2下断点，F9跳出循环）。

循环2

继续执行看到第二个循环：解压缩循环

010153E4	. 75 F7	jnz short notepad_.01015300
010153E6	. ^ E9 63FFFFFF	jmp notepad_.0101534E
010153EB	90	nop
010153EC	> 8B 02	mov eax,dword ptr ds:[edx]
010153EE	. 83C2 04	add edx,0x4
010153F1	. 89 07	mov dword ptr ds:[edi],eax
010153F3	. 83C7 04	add edi,0x4
010153F6	. 83E9 04	sub ecx,0x4
010153F9	. ^ 77 F1	ja short notepad_.010153EC
010153FB	. 01CF	add edi,ecx
010153FD	. ^ E9 4CFFFFFF	jmp notepad_.0101534E

循环2

继续执行看到第二个循环：解压缩循环

01015348	>	8A 06	mov al,byte ptr ds:[esi]
0101534A	.	46	inc esi
0101534B	.	88 07	mov byte ptr ds:[edi],al
0101534D	.	47	inc edi
.....			
010153DD	>	8A 02	mov al,byte ptr ds:[edx]
010153DF	.	42	inc edx
010153E0	.	88 07	mov byte ptr ds:[edi],al
010153E2	.	47	inc edi
.....			
010153EC	>	8B 02	mov eax,dword ptr ds:[edx]
010153EE	.	83 C2 04	add edx,0x4
010153F1	.	89 07	mov dword ptr ds:[edi],eax
010153F3	.	83 C7 04	add edi,0x4

先从ESI所指的第二个节区(UPX1)地址中依次读取值，经过适当的运算解压缩后，将值写入
EDI所指的第一个节区 (UPX0) 地址(01007000)。

在01015402处设置断点，F9跳出循环。

循环3

该循环代码用于恢复源代码的CALL/JMP指令
(操作码E8/E9) 的desitination地址。

01015405	B9 32010000	mov ecx,132		
0101540A	8A07	mov al,byte ptr ds:[edi]	小循环	notepad_.01001831
0101540C	47	inc edi		
0101540D	2C E8	sub al,0E8		
0101540F	3C 01	cmp al,1		
01015411	77 F7	ja short notepad_.0101540A		
01015413	803F 01	cmp byte ptr ds:[edi],1		
01015416	75 F2	jnz short notepad_.0101540A		
01015418	8B07	mov eax,dword ptr ds:[edi]		
0101541A	8A5F 04	mov bl,byte ptr ds:[edi+4]		
0101541D	66:C1E8 08	shr ax,8		
01015421	C1C0 10	rol eax,10		
01015424	86C4	xchg ah,al		
01015426	29F8	sub eax,edi		notepad_.01001831
01015428	80EB E8	sub bl,0E8		
0101542B	01F0	add eax,esi		notepad_.01001000
0101542D	8907	mov dword ptr ds:[edi],eax		
0101542F	83C7 05	add edi,5		
01015432	88D8	mov al,bl		
01015434	E2 D9	loopd short notepad_.0101540F		https://blog.csdn.net/diamond_biu

01015436地址处设置断点运行后即可跳出循环。

循环4
该循环用于恢复IAT。

设置EDI=01014000，它指向存储着源文件调用的API函数名称的字符串

01015436	8DBE 00300100	lea edi,dword ptr ds:[esi+13000]	
0101543C	8B07	mov eax,dword ptr ds:[edi]	EDI=01014000，指向第二节区，该区域保存着原
0101543E	09C0	or eax,eax	notepad调用的API函数名称字符串。
01015440	74 3C	je short notepad_.0101547E	
01015442	8B5F 04	mov ebx,dword ptr ds:[edi+4]	
01015445	8D8430 04BE010	lea eax,dword ptr ds:[eax+esi+1BE04]	
0101544C	01F3	add ebx,esi	notepad_.01001000
0101544E	50	push eax	ADVAPI32.RegSetValueExW
0101544F	83C7 08	add edi,8	
01015452	FF96 CCBE0100	call dword ptr ds:[esi+1BECC]	kernel32.LoadLibraryA
01015458	95	xchg eax,ebp	ADVAPI32.77850000
01015459	8A07	mov al,byte ptr ds:[edi]	
0101545B	47	inc edi	notepad_.010143FA
0101545C	08C0	or al,al	
0101545E	74 DC	je short notepad_.0101543C	
01015460	89F9	mov ecx,edi	notepad_.010143FA
01015462	57	push edi	notepad_.010143FA
01015463	48	dec eax	ADVAPI32.RegSetValueExW
01015464	F2:AE	repne scas byte ptr es:[edi]	
01015466	55	push ebp	kernel32.77850000
01015467	FF96 D0BE0100	call dword ptr ds:[esi+1BED0]	kernel32.GetProcAddress
0101546D	09C0	or eax,eax	ADVAPI32.RegSetValueExW
0101546F	74 07	je short notepad_.01015478	
01015471	8903	mov dword ptr ds:[ebx],eax	将地址输入ebx所指的IAT区域
01015473	83C3 04	add ebx,4	

地址	HEX 数据																ASCII	
01014000	24	01	00	00	8C	00	00	00	01	47	65	74	43	75	72	72	\$\$.?..GetCurrent	
01014010	65	6E	74	54	68	72	65	61	64	49	64	00	01	47	65	74	entThreadId.GetCurrent	
01014020	54	69	63	6B	43	6F	75	6E	74	00	01	51	75	65	72	79	TickCount.Query	
01014030	50	65	72	66	6F	72	6D	61	6E	63	65	43	6F	75	6E	74	PerformanceCount	
01014040	65	72	00	01	47	65	74	4C	6F	63	61	6C	54	69	6D	65	er.GetLocalTime	
01014050	00	01	47	65	74	55	73	65	72	44	65	66	61	75	6C	74	.GetUserDefault	
01014060	4C	43	49	44	00	01	47	65	74	44	61	74	65	46	6F	72	LCID.GetDateFor	
01014070	6D	61	74	57	00	01	47	65	74	54	69	6D	65	46	6F	72	matW.GetTimeFor	
01014080	6D	61	74	57	00	01	47	6C	6F	62	61	6C	4C	6F	63	6B	matW.GlobalLock	
01014090	00	01	47	6C	6F	62	61	6C	55	6E	6C	6F	63	6B	00	01	.GlobalUnlock.	
010140A0	47	65	74	46	69	6C	65	49	6E	66	6F	72	6D	61	74	69	GetFileInformati	
010140B0	6F	6E	42	79	48	61	6E	64	6C	65	00	01	43	72	65	61	onByHandle.Crea	

UPX压缩源文件时，会分析其IAT，提取程序中调用的API名称列表，形成API名称字符串。因此在恢复时，就需要用这些API名称字符串调用GetProcAddress()函数，获得API起始地址，再将起始地址输入IAT区域。

全部解压缩完成后(解压结束后执行到： 01015484)
， 程序的控制会返回OEP处。

010154AD	61	popad	
010154AE	8D4424 80	lea eax,dword ptr ss:[esp-80]	
010154B2	6A 00	push 0	
010154B4	39C4	cmp esp,eax	
010154B6	^ 75 FA	jnz short notepad_.010154B2	
010154B8	83EC 80	sub esp,-80	
010154BB	- E9 DD1EFFFF	jmp notepad_.0100739D	

POPAD与PUSHAD对应。最终跳转到0100739D处， 即为原notepad.exe的EP。

快速查找UPX OEP的方法——在栈中设置硬件断点

UPX压缩器的特征之一是：其EP代码被包含在PUSHAD/POPAD指令之间，并且跳转到OEP的JMP指令紧接着出现在POPAD指令之后。在栈中设置硬件断点的方法利用了UPX的上述特性。在执行PUSHAD指令后，查看栈。可以看到EAX~EDI寄存器的值依此入栈。

```
寄存器 (FPU)
EAX 00000000
ECX 0006FFB0
EDX 7C92E4F4 ntdll.KiFastSystemCallRet
EBX 7FFD5000
ESP 0006FFA4 ASCII "jk"
EBP 0006FFF0
ESI 0012B880
EDI 006B6A20

EIP 01015331 notepad_.01015331

C 0 ES 0023 32位 0(FFFFFFFF)
P 1 CS 001B 32位 0(FFFFFFFF)
A 0 SS 0023 32位 0(FFFFFFFF)
Z 1 DS 0023 32位 0(FFFFFFFF)
S 0 FS 003B 32位 7FFDF000(FFF)
T 0 GS 0000 NULL
D 0
0 0 LastErr ERROR_NO_IMPERSONATION_TOKEN (000)

EFL 00000246 (NO,NB,E,BE,NS,PE,GE,LE)

ST0 empty +UNORM 401C 7C934029 7C99D600
ST1 empty +UNORM 007A 00000000 0012BC98
ST2 empty +UNORM 0477 000C0D0C 77D30B05
ST3 empty +UNORM 0002 0000000D 00000000
```

```
吾爱破解 - notepad_upx.exe - [LCG - 主线程, 模块 - notepad_]
文件(F) 查看(V) 调试(D) 插件(P) 选项(O) 窗口(W) 帮助(H) [+] 快捷菜单 Tools BreakPoint->
暂停
01015330 $ 60 pushad
01015331 . BE 00100101 mov esi,notepad_.01011000
01015336 . 8DBE 0000FFFF lea edi,dword ptr ds:[esi-0x10000]
0101533C . 57 push edi
0101533D . 83CD FF or ebp,-0x1
01015340 . EB 10 jmp short notepad_.01015352
01015342 90 nop
01015343 nop
01015344 nop
01015345 nop
01015346 nop
01015347 nop
01015348 nop
0101534A nop
0101534B nop
0101534C nop
0101534D nop
0101534E nop
01015350 jnz short notepad_.01015359
01015352 mov ebx,dword ptr ds:[esi]
01015354 sub esi,-0x4
01015357 adc ebx,ebx
01015359 jb short notepad_.01015348
0101535B mov eax,0x1
01015360 add ebx,ebx
01015362 inz short notepad_.0101536B

地址 H
界面选项
```

执行程序(F9)，可以看到运行完POPAD处停止（设置硬件断点的指令执行完毕后才暂停调试），可以看到它的下方就是跳转到OEP的JMP指令。

010154AD	. 61	popad	
010154AE	. 8D4424 80	lea eax,dword ptr ss:[esp-80]	
010154B2	> 6A 00	push 0	
010154B4	. 39C4	cmp esp,eax	kernel32
010154B6	.^ 75 FA	jnz short notepad_.010154B2	
010154B8	. 83EC 80	sub esp,-80	
010154BB	.- E9 DD1EFFFF	jmp notepad_.0100739D	

(RebPE) 按ALT+M, 打开内存镜象, 找到第一个.data,按F2,设置内存访问断点,
然后按SHIFT+F9运行到断点, 接着再按ALT+M, 打开内存镜象, 找到.text,
设置内存访问断点! 然后按SHIFT+F9, 直接到达程序OEP(00401130)

00340000	00041000				Map	R	R
00390000	00001000				Priv	RW	RW
003A0000	00003000				Priv	RW	RW
00400000	00001000	RebPE		PE 文件头	Image	R	RWE
00401000	00004000	RebPE	.text				
00405000	00001000	RebPE	.rdat				
00406000	00003000	RebPE	.data				
00409000	0000A000	RebPE	.rsrce				
00413000	00007000	RebPE	.pedi				
7C800000	00001000	kernel32					
7C801000	00084000	kernel32	.text				
7C885000	00005000	kernel32	.data				
7C88A000	0008E000	kernel32	.rsrce				
7C918000	00006000	kernel32	.reloc				
7C920000	00001000	ntdll					
7C921000	0007A000	ntdll	.text				
7C99B000	00005000	ntdll	.data				

刷新

在反汇编窗口中查看
在 CPU 数据窗口中查看
数据
查找

Ctrl+B

在访问上设置中断
F2

设置内存访问断点 (A)

脱壳

在OEP处右键-Dump debugged process (52pojie OD中为“用Ollydump脱壳调试进程”)

0100739D6A 70push 0x70

0100739F68 98180001push notepad_.01001898

010073A4

010073A9

010073AB

010073AC

010073B2

010073B4

010073B9

010073BB

010073BE

010073C0

010073C6

010073C8

010073CC

010073D1

010073D3

010073D8

010073DA

010073DD

010073DF

010073E6

010073E8

010073EA

010073F0

OllyDump - notepad_upx.exe

起始地址:1000000大小:1E000

入口点地址:15330->修正为:739D获取EIP作为OEP取消

代码基址:11000数据基址:16000

☒ 在脱壳镜像中修正物理地址和物理大小

Sec...	Virtual...	Virtual...	Raw Size	Raw Offset	Characteristi...
UPX0	00010000	00001000	00010000	00001000	E0000080
UPX1	00005000	00011000	00005000	00011000	E0000040
.rsrc	00008000	00016000	00008000	00016000	C0000040

☒ 重建输入表

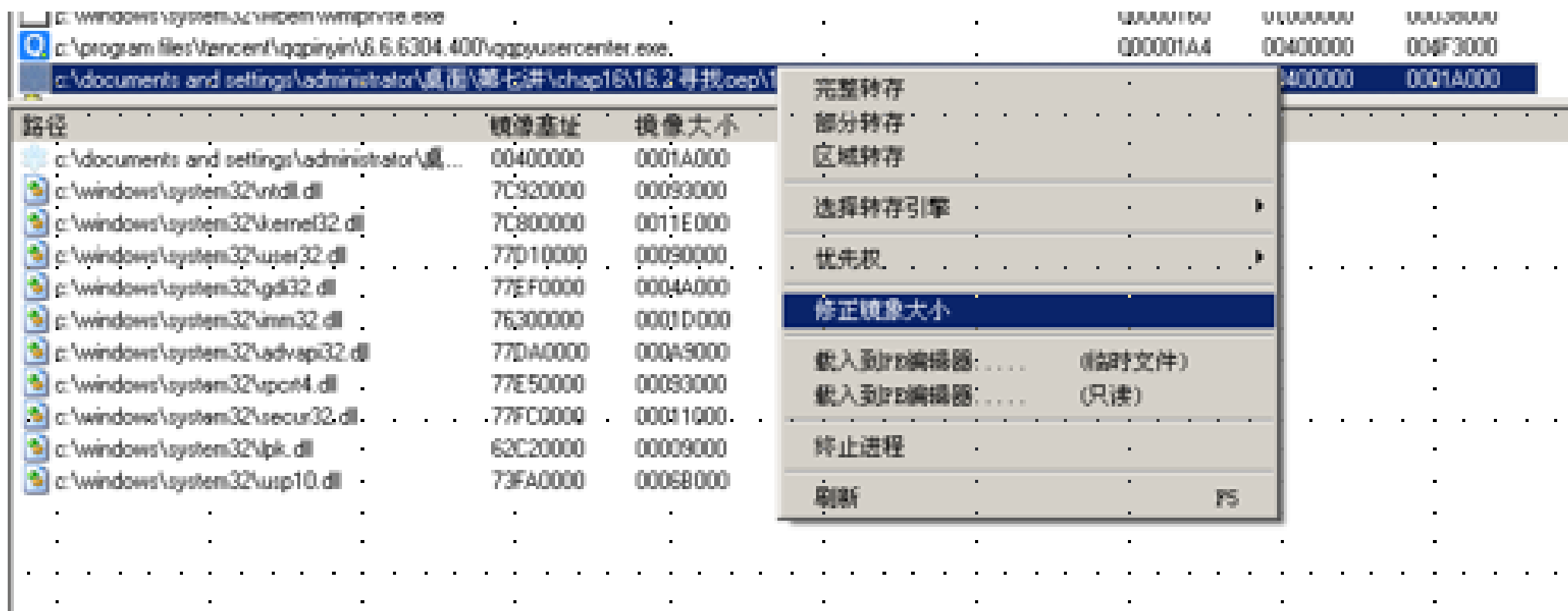
- ☒ 方式1 : 在内存镜像中搜索 JMP[API] | CALL[API]
- ☐ 方式2 : 在脱壳文件中搜索 DLL & API 名称

☆ 汉化:dyk158 ☆ [05.04.21]

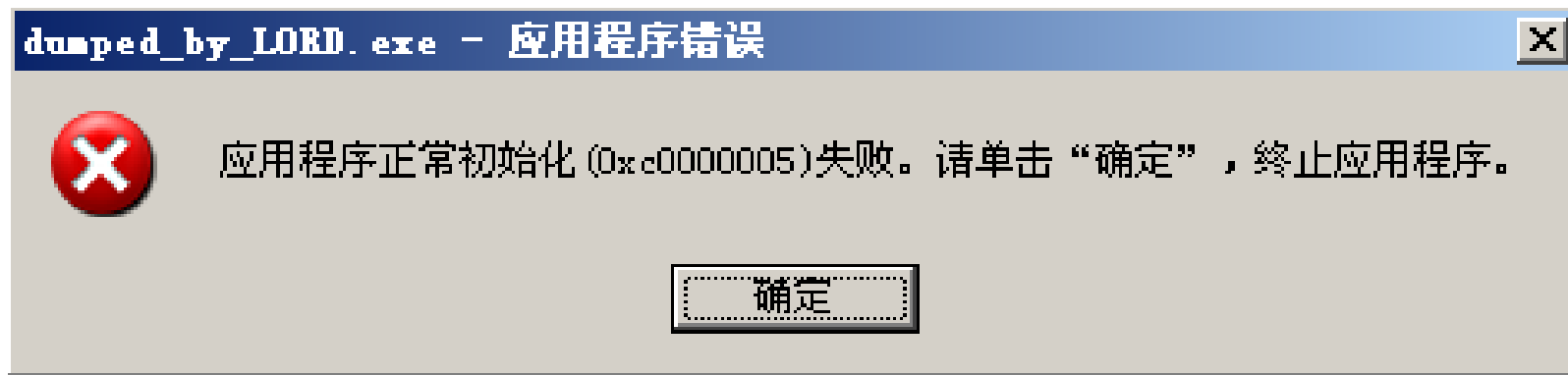
EB 0E| 0000 500F notepad_.01001898

《加密与解密》 第16章

- 先找到入口点
- 然后用LordPE dump,(先修正大小, 再进行完整转存)



不能正常运行



Import REC重建输入表

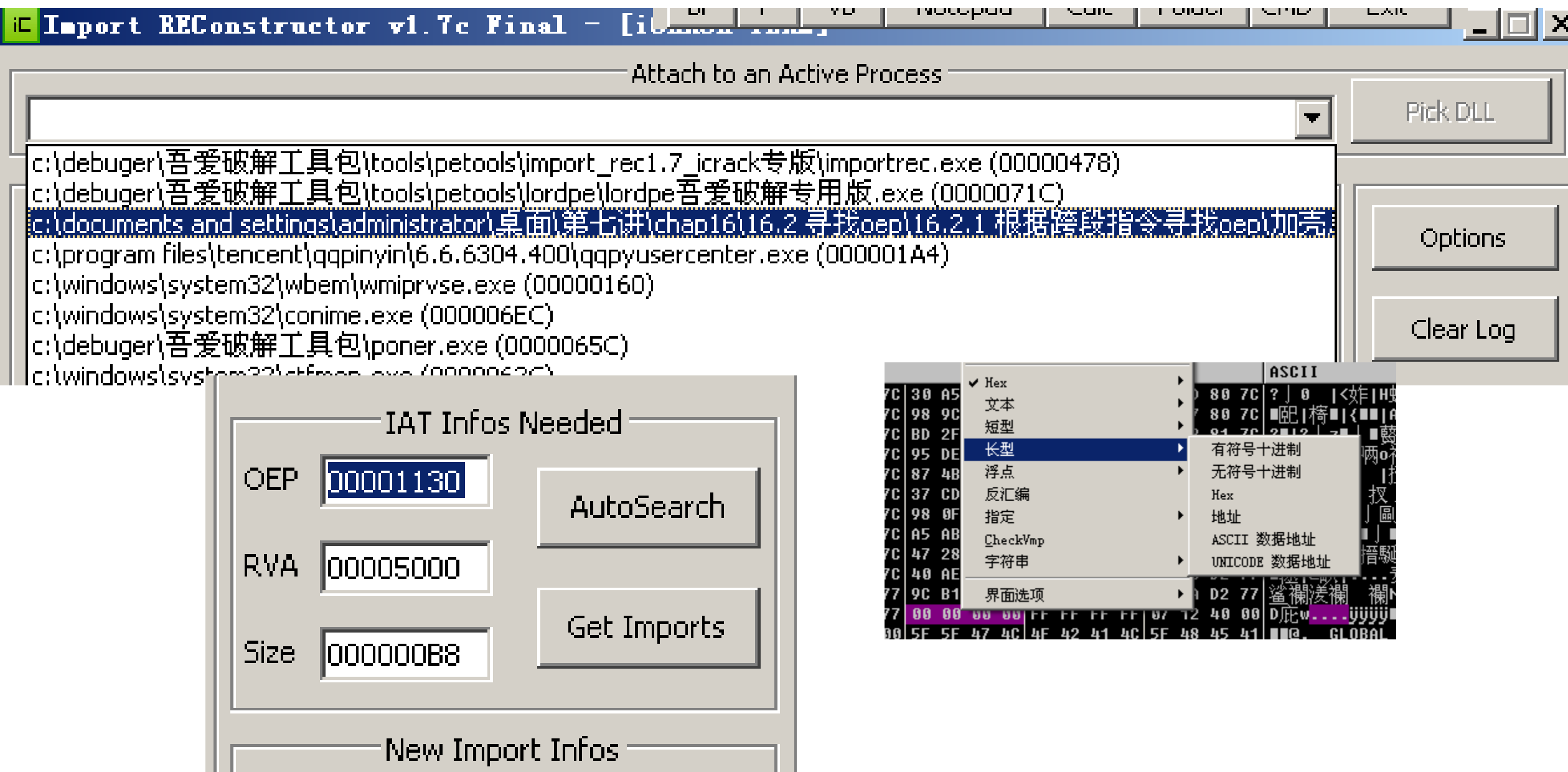
- 运行这个软件前我们要确保几个事情：
- 目标文件已经被像我们上面那样给Dump下来了。
- 目标文件正在运行
- 我们事先得知道目标程序的真正OEP或者IAT的偏移量和大小。

所以如果完成了上面的Dump的步骤之后，我们再将原来的没有Dump的程序用OD打开，然后跟踪到OEP处：

暂停				l e m t w h c p k b r ... s												BP	P	v
00401130	.	55	push ebp															
00401131	.	8BEC	mov ebp,esp															
00401133	.	6A FF	push -0x1															
00401135	.	68 B8504000	push RebPE.004050B8															
0040113A	.	68 FC1D4000	push RebPE.00401DFC															
0040113F	.	64:A1 0000000	mov eax,dword ptr fs:[0]															
00401145	.	50	push eax															
00401146	.	64:8925 0000	mov dword ptr fs:[0],esp															
0040114D	.	83EC 58	sub esp,0x58															
00401150	.	53	push ebx															
00401151	.	56	push esi															
00401152	.	57	push edi															
00401153	.	8965 E8	mov [local.6],esp															
00401156	.	FF15 2850400	call dword ptr ds:[0x405028]	kernel32.GetVersion														

SE 处理程序安装

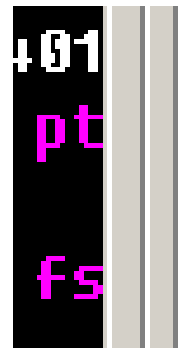
然后我们打开软件，在下拉栏里面找到目标进程：



然后再点击IAT自动搜索，让软件自动搜索IAT的偏移量和大小：

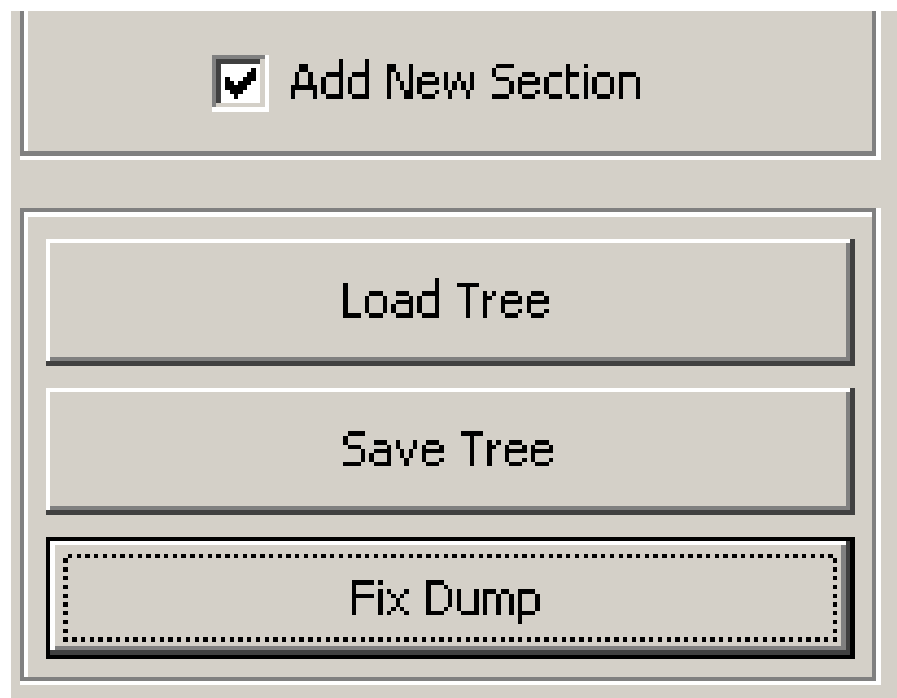


• 点击获取导入表：



+ kernel32.dll FThunk:00005000 NbFunc:26 (decimal:38) valid:YES
+ user32.dll FThunk:0000509C NbFunc:6 (decimal:6) valid:YES

最后就到了修复程序的时候了，在下面这个地方的复选框中选中添加新节区：



`dumped_by_LORD.exe`

实验内容：

- 1.对notepad_upx.exe进行脱壳；初步掌握EXE脱壳一般流程和常用方法。
- 2.对RebPe.exe进行脱壳；
- 3.对55.exe进行脱壳。

大作业题目

- 1. Android逆向分析
- 2. 序列号保护机制和破解
- 3. 加壳脱壳
- 4. 恶意代码分析
- 5. CTF逆向题目
- 6. 自拟题目：与软件逆向相关均可

- **【要求】** 1.不超过3人一组，要求有可演示的结果。
- 2.设计报告按西南科技大学学报自然科学版论文格式排版，参考文献不少于5篇。
- 3.不能用其它课程的设计报告来交，一经发现，本部分记为零分。
- **【评分】** 总分100，分组的基础分20分，其余按小组贡献给分。
- **【提交时间】** 本学期5月14号开始提交，具体地点请看群消息。